

5 📖 Documentation_Docker_Swarm

🐳 Projet : Docker Swarm

Conception et déploiement d'un cluster Docker Swarm de 7 machines virtuelles Debian, organisé en 3 managers Raft et 3 workers, complété par un serveur NFS pour la persistance des données. Déploiement des 5 services applicatifs requis (Registry, MariaDB, PHP, Nginx, VSCode) et ajout d'une stack de supervision complète (Portainer, Prometheus, Grafana) pour superviser les 6 nœuds en temps réel. La résilience du cluster a été validée par 3 tests PCA/PRA qui démontrent successivement le self-healing au niveau conteneur, la redistribution automatique en cas de perte d'un nœud worker, et l'élection automatique d'un nouveau leader Raft en cas de défaillance du manager principal.

Sommaire :

- [1 🎓 - Présentation du projet](#)
 - [2 💻 - Préparation de l'infrastructure](#)
 - [3 🐳 - Installation de Docker & Création du Cluster Swarm](#)
 - [4 💾 - Configuration du stockage NFS](#)
 - [5 🚀 - Déploiement des services : Registry, MariaDB, PHP, Nginx & VSCode](#)
 - [6 👁️ - Supervision et monitoring](#)
 - [7 🧪 - Test PCA/PRA](#)
-

1 🎓 - Présentation du projet

Dans un contexte où la disponibilité des services numériques est devenue un enjeu stratégique majeur, ce projet a pour objectif de concevoir et déployer une plateforme d'orchestration de conteneurs basée sur **Docker Swarm**, dans une optique de **continuité et de reprise d'activité**.

L'infrastructure repose sur un ensemble de machines virtuelles **Debian**, organisées en cluster Swarm selon une architecture manager/workers, complétée par un serveur de

stockage persistant NFS. Cette approche permet de garantir la résilience des services déployés : en cas de défaillance d'un nœud, le cluster redistribue automatiquement les conteneurs sur les machines disponibles.

Les services hébergés au sein du cluster couvrent l'ensemble de la chaîne applicative : un registry Docker interne pour la gestion des images, une base de données MariaDB, un serveur applicatif PHP, un reverse proxy Nginx pour l'exposition des services, ainsi qu'un environnement de développement collaboratif VSCode Server.

Au-delà du déploiement technique, ce projet met en œuvre une démarche **DevOps** complète : infrastructure définie sous forme de code (fichiers de stack), procédures de tests documentées, simulation de pannes et validation des mécanismes de PCA et PRA.

Ce projet constitue ainsi une mise en situation concrète des compétences d'administration système, de virtualisation et de supervision d'infrastructure dans un environnement conteneurisé.

1 — Préparation de l'infrastructure : créer et configurer les VMs Debian (manager, workers, NFS), configurer le réseau entre elles, activer SSH, définir les IPs fixes.

2 — Installation de Docker Engine : installer Docker sur chaque VM, ajouter les utilisateurs au groupe docker, vérifier que le service tourne.

3 — Création du cluster Swarm : initialiser le Swarm sur le manager, faire rejoindre les workers avec le token, vérifier l'état du cluster.

4 — Configuration du stockage NFS : configurer la VM NFS, créer les exports, monter les partages sur les workers, créer les volumes Docker pointant sur NFS.

5 — Déploiement des services : écrire le fichier `stack.yml` et déployer Registry, MariaDB, PHP, Nginx et VSCode Server sur le cluster.

6 — Supervision et monitoring : déployer Portainer pour la gestion visuelle, Prometheus et Grafana pour les métriques et alertes.

7 — Tests PCA/PRA et documentation : simuler des pannes, vérifier le comportement du cluster, rédiger les procédures.

 [Sommaire :](#)

2 - Préparation de l'infrastructure

Créer et configurer les VMs Debian (manager, workers, NFS), configurer le réseau entre elles, activer SSH, définir les IPs fixes.

2.1 Schéma de l'infrastructure :

VM	CPU	RAM	Disque	Rôle
Manager 1 Nœud principal Raft	2 vCPU	1 Go	20 Go	Raft leader pas de conteneurs
Manager 2 Nœud Raft secondaire	2 vCPU	1 Go	20 Go	Raft follower pas de conteneurs
Manager 3 Nœud Raft secondaire	2 vCPU	1 Go	20 Go	Raft follower pas de conteneurs
Worker 1 Nginx — PHP — Registry	2 vCPU	2 Go	20 Go	Services web + images registry
Worker 2 MariaDB — VSCode	2 vCPU	2 Go	20 Go	BDD + dev environnement
Worker 3 Réplicas — redondance	2 vCPU	2 Go	20 Go	PCA/PRA renforcé bascule test panne
Serveur NFS Stockage partagé	1 vCPU	1 Go	50 Go	Volumes Docker données persistantes
TOTAL cluster				
	13 vCPU	10 Go RAM	170 Go	

2.1 Vérification des ressources disponibles :

RAM :

```
Get-CimInstance Win32_OperatingSystem | Select-Object
TotalVisibleMemorySize, FreePhysicalMemory
```

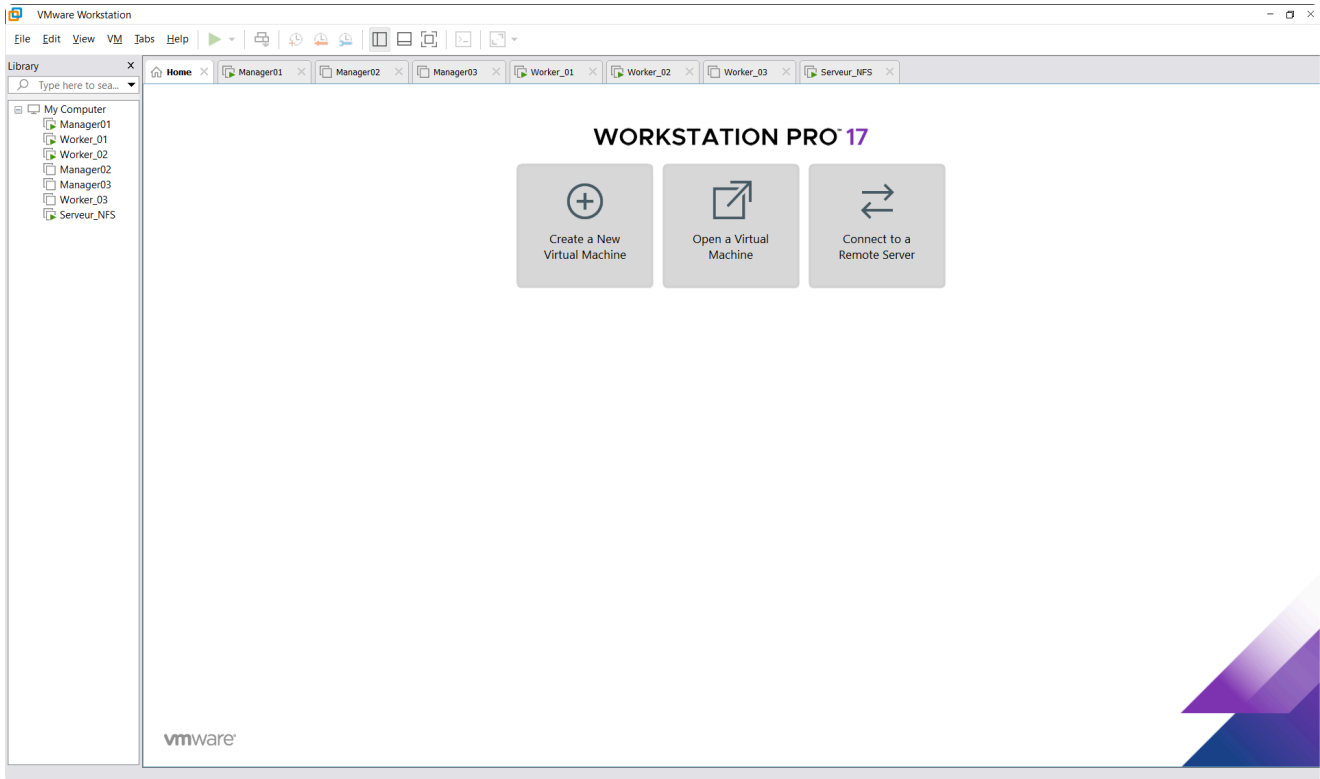
CPU :

```
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors
```

Disque :

Get-PSDrive C | Select-Object Used, Free

2.3 Création de 7 machines virtuelles :



2.4 Attribution des IP statiques : nano /etc/network/interfaces

```
GNU nano 8.4 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
#allow-hotplug ens33
#iface ens33 inet dhcp

auto ens33
iface ens33 inet static
    address 192.168.163.10
    netmask 255.255.255.0
    gateway 192.168.163.2
    dns-nameservers 8.8.8.8
```

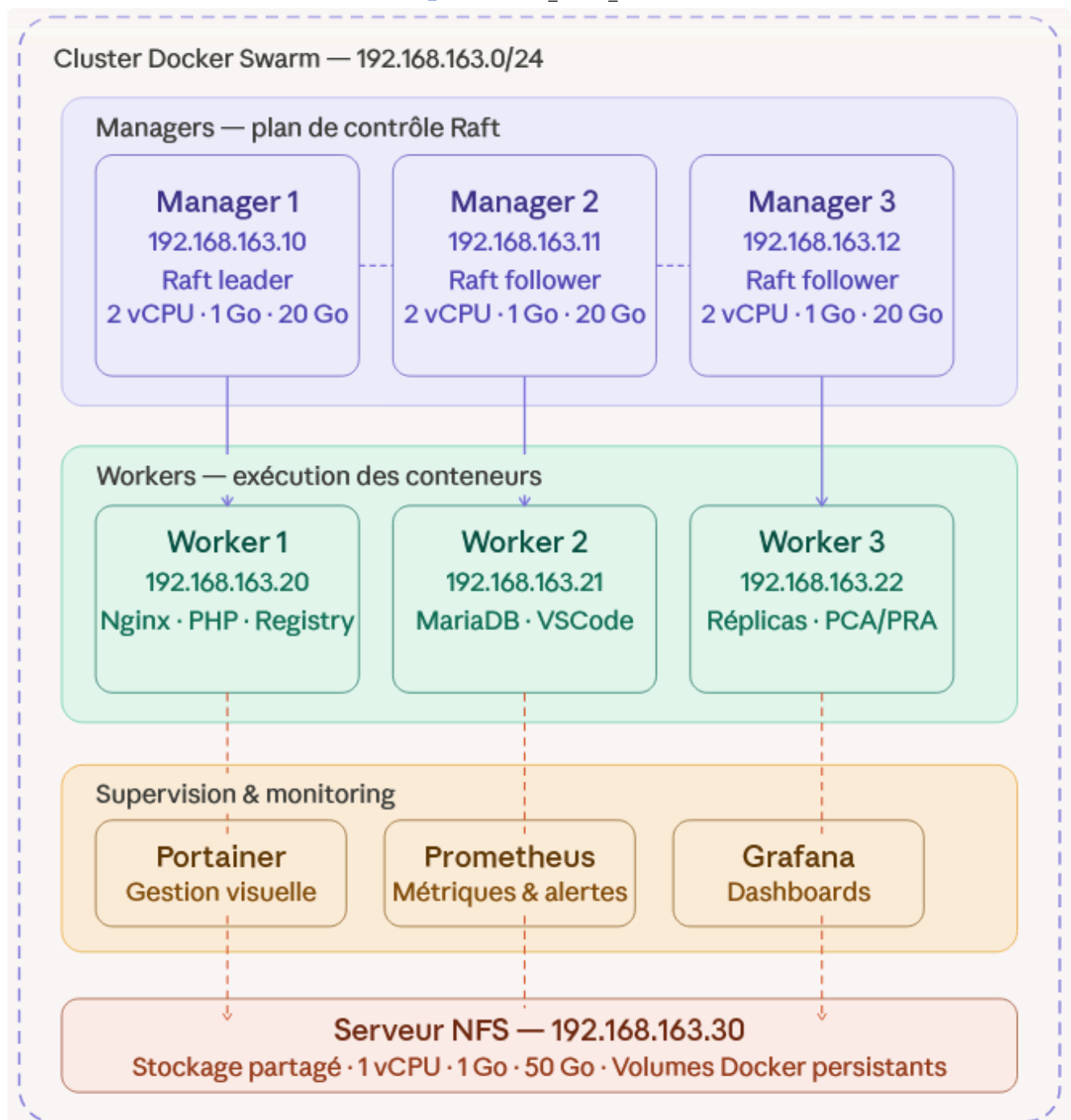
```

root@manager01:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:64:ea:56 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2964ea56
    inet 192.168.163.10/24 brd 192.168.163.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe64:ea56/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@manager01:~# _

```

2.5 Plan d'adressage IP :

1. manager01
192.168.163.10
2. manager02
192.168.163.11
3. manager03
192.168.163.12
4. worker01
192.168.163.20
5. worker02
192.168.163.21
6. worker03
192.168.163.22
7. servernfs
192.168.163.30
8. Gateway
192.168.163.2
9. Netmask
255.255.255.0 --> 254 addresses utilisables



2.6 - Installation du service SSH

```

root@manager01:~# systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-03-31 09:02:43 CEST; 2min 37s ago
   Invocation: cbdb84febbf247ff80272d7e4c69c3f0
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 1133 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
  Main PID: 1138 (sshd)
    Tasks: 1 (limit: 1047)
   Memory: 2.6M (peak: 3.3M)
      CPU: 23ms
   CGroup: /system.slice/ssh.service
           └─1138 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

mars 31 09:02:43 manager01 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
mars 31 09:02:43 manager01 sshd[1138]: Server listening on 0.0.0.0 port 22.
mars 31 09:02:43 manager01 sshd[1138]: Server listening on :: port 22.
mars 31 09:02:43 manager01 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
root@manager01:~#

```

2.7 - Ping entre les différentes machines

```

root@manager01:~# ping 192.168.163.20
PING 192.168.163.20 (192.168.163.20) 56(84) bytes of data.
 64 bytes from 192.168.163.20: icmp_seq=1 ttl=64 time=1.35 ms
 64 bytes from 192.168.163.20: icmp_seq=2 ttl=64 time=0.555 ms
 64 bytes from 192.168.163.20: icmp_seq=3 ttl=64 time=0.422 ms
 64 bytes from 192.168.163.20: icmp_seq=4 ttl=64 time=0.541 ms
 64 bytes from 192.168.163.20: icmp_seq=5 ttl=64 time=0.652 ms
 64 bytes from 192.168.163.20: icmp_seq=6 ttl=64 time=0.767 ms
^C
--- 192.168.163.20 ping statistics ---
 6 packets transmitted, 6 received, 0% packet loss, time 5085ms
 rtt min/avg/max/mdev = 0.422/0.714/1.350/0.303 ms
root@manager01:~#

```

[↑ Sommaire :](#)

3 - Installation de Docker & Création du Cluster Swarm

Installation de Docker sur les sept machines virtuelles.

3.1 Installation des dépendances avec la commande : `apt install -y ca-certificates curl gnupg`

Une dépendance est un logiciel **nécessaire au fonctionnement d'un autre logiciel**.

```
root@manager01:~# apt install -y ca-certificates curl gnupg
ca-certificates est déjà la version la plus récente (20250419).
curl est déjà la version la plus récente (8.14.1-2+deb13u2).
gnupg est déjà la version la plus récente (2.4.7-21+deb13u1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@manager01:~# _
```

3.2 Ajoût de la clé GPG docker et création du dossier keyrings avec la commande : `install -m 0755 -d /etc/apt/keyrings`

Une **clé GPG** (GNU Privacy Guard) est un fichier cryptographique qui permet de **vérifier l'authenticité** et **l'intégrité** d'un fichier, garantissant qu'il provient bien de son auteur officiel. Ici la clé GPG nous garantit d'installer la version officiel de Docker et pas un version modifiée ou malveillante.

```
root@manager01:~# install -m 0755 -d /etc/apt/keyrings
root@manager01:~#
```

Qui	Permissions	Peut faire
root	7 (rwx)	Lire, écrire, exécuter
groupe	5 (r-x)	Lire, exécuter
autres	5 (r-x)	Lire, exécuter

Pourquoi ces permissions ?

- root peut écrire les clés GPG dans ce dossier
- Tout le monde peut lire les clés ce qui nécessaire pour que apt puisse les vérifier
- Personne d'autre que root ne peut modifier les clés

3.3 Téléchargement de la clé officielle de Docker avec la commande : `curl -fsSL https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg`

Cette commande télécharge la clé GPG officielle de Docker depuis internet et la convertit dans un format que apt peut utiliser pour vérifier les paquets Docker.

```
root@manager01:~# curl -fsSL https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
root@manager01:~# _
```


3.4 Attribution des permissions sur la clé avec la commande : `chmod a+r /etc/apt/keyrings/docker.gpg`

```
root@manager01:~# chmod a+r /etc/apt/keyrings/docker.gpg
root@manager01:~#
```

a → all (tout le monde) + → ajouter r → read (lire)

3.5 Vérification des droits sur le dossier `/etc/apt/keyrings` avec la commande : `ls -la /etc/apt/keyrings`

```
root@manager01:~# ls -la /etc/apt/keyrings/
total 12
drwxr-xr-x 2 root root 4096 31 mars 09:21 .
drwxr-xr-x 9 root root 4096 30 mars 15:15 ..
-rw-rw-r-- 1 root root 2760 31 mars 09:21 docker.gpg
root@manager01:~# _
```

3.6 Ajoût du dépôt Docker avec la commande : `echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/debian trixie stable" | tee /etc/apt/sources.list.d/docker.list > /dev/null`

Ajoût de dépôt bookworm avec la commande : `echo "deb http://deb.debian.org/debian bookworm main" | tee /etc/apt/sources.list.d/bookworm.list`

Pourquoi ajouter bookworm sur Debian 13 ?

Debian 13 Trixie a des conflits de versions avec Docker (`libxtables12` / `iptables`). En ajoutant le dépôt Debian 12, on peut forcer l'installation des versions compatibles de ces paquets.

Mettre à jour avec la commande : `apt update`

3.7 Résolution du conflit débien 13 avec les commandes :

En raison d'incompatibilités entre Debian 13 Trixie et Docker, on force l'installation de versions plus anciennes de `libxtables12` et `iptables` depuis Debian 12 pour rendre l'installation de Docker possible.

- `apt-get install -y --allow-downgrades libxtables12=1.8.9-2 -t bookworm`

- **apt-get install -y --allow-downgrades iptables=1.8.9-2 -t bookworm**

3.8 Téléchargement et execution du script docker pour procéder à l'installation avec les commandes :

Le script officiel Docker gère automatiquement toutes les dépendances et la configuration — c'est la méthode recommandée par Docker Inc. pour les installations sur des systèmes non standards comme Debian 13.

- **curl -fsSL <https://get.docker.com> -o get-docker.sh**

- **sh get-docker.sh**

3.9 Activation de docker au démarrage avec la commande : **systemctl enable docker**

3.10 Vérification avec les commandes **docker --version** et **systemctl status docker**

```
root@manager01:~# docker --version
Docker version 29.3.1, build c2be9cc
root@manager01:~# _
```

```
root@manager01:~# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-03-31 11:24:39 CEST; 42s ago
   Invocation: d96d1563fb2c420898728ebbc7607d0b
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 2534 (dockerd)
       Tasks: 9
      Memory: 78.8M (peak: 81.7M)
         CPU: 630ms
    CGroup: /system.slice/docker.service
            └─2534 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

mars 31 11:24:38 manager01 dockerd[2534]: time="2026-03-31T11:24:38.233047589+02:00" level=info msg="Restoring containers: start."
mars 31 11:24:38 manager01 dockerd[2534]: time="2026-03-31T11:24:38.356575604+02:00" level=info msg="Deleting nftables IPv4 rules" error="exit status 1" output=
mars 31 11:24:38 manager01 dockerd[2534]: time="2026-03-31T11:24:38.376444207+02:00" level=info msg="Deleting nftables IPv6 rules" error="exit status 1" output=
mars 31 11:24:39 manager01 dockerd[2534]: time="2026-03-31T11:24:39.005402826+02:00" level=info msg="Loading containers: done."
mars 31 11:24:39 manager01 dockerd[2534]: time="2026-03-31T11:24:39.023049504+02:00" level=info msg="docker daemon" commit=f78c987 containerd-snapshotter=true
mars 31 11:24:39 manager01 dockerd[2534]: time="2026-03-31T11:24:39.023299560+02:00" level=info msg="Initializing buildkit"
mars 31 11:24:39 manager01 dockerd[2534]: time="2026-03-31T11:24:39.051814135+02:00" level=info msg="Completed buildkit initialization"
mars 31 11:24:39 manager01 dockerd[2534]: time="2026-03-31T11:24:39.072336949+02:00" level=info msg="Daemon has completed initialization"
mars 31 11:24:39 manager01 dockerd[2534]: time="2026-03-31T11:24:39.072450656+02:00" level=info msg="API listen on /run/docker.sock"
mars 31 11:24:39 manager01 systemd[1]: Started docker.service - Docker Application Container Engine.
lines 1-23/23 (END)
```

3.11 Initialisation du Swarm Manager 01 avec la commande : **docker swarm init --advertise-addr 192.168.163.10** et récupération du du token "worker"

Récupération du token worker avec la commande : **docker swarm join-token worker**

```
root@manager01:~# docker swarm init --advertise-addr 192.168.163.10
Swarm initialized: current node (co8cqeptkhkjsrhnckyg847y) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-db0c1uquiy697ruvvr06g2672 192.168.163.10:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
root@manager01:~#
```

Récupération du token manager avec la commande : **docker swarm join-token manager**

```
root@manager01:~# docker swarm join-token manager
To add a manager to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-38kuz20uooooh1wnywurs2mwd 192.168.163.10:2377

root@manager01:~#
```

3.12 Ajout des Managers 2 et 3 avec la commande : **docker swarm join --token "TOKEN MANAGER" 192.168.163.10:2377**

```
root@manager02:~# docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-38kuz20uooooh1wnywurs2mwd 192.168.163.10:2377
This node joined a swarm as a manager.
root@manager02:~#
```

```
root@manager03:~# docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-38kuz20uooooh1wnywurs2mwd 192.168.163.10:2377
This node joined a swarm as a manager.
root@manager03:~# |
```

3.13 Ajout des Workers 1, 2 et 3 avec la commande : **docker swarm join --token "TOKEN WORKER" 192.168.163.10:2377**

```
root@worker01:~# docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-db0c1uquiy697ruvvr06g2672 192.168.163.10:2377
This node joined a swarm as a worker.
root@worker01:~#
```

```
root@worker02:~# docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-db0c1uquiy697ruvvr06g2672 192.168.163.10:2377
This node joined a swarm as a worker.
root@worker02:~#
```

```
root@worker03:~# docker swarm join --token SWMTKN-1-0n3jm53n1e0wgv9ala3ktz37266t30vy1fl6puxn3f1525hali-db0c1uquiy697ruvvr06g2672 192.168.163.10:2377
This node joined a swarm as a worker.
root@worker03:~# |
```

3.14 Vérification de l'état du Cluster avec la commande : `docker node ls`

```
root@manager01:~# docker node ls
ID                                HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqepthkjserhndekyg847y *     manager01   Ready     Active           Leader             29.3.1
y293owicg2mi7czcfltcquwmw      manager02   Ready     Active           Reachable          29.3.1
ux86nv0shfyvaf0jxkjjt91u5      manager03   Ready     Active           Reachable          29.3.1
z12wbrmij3sr3kssnurcp17o3      worker01    Ready     Active           -                  29.3.1
8ifbk2m8yvhmtalyfl1bg0a2t      worker02    Ready     Active           -                  29.3.1
bggkrw9zvodb86uu8ja5bq1nl      worker03    Ready     Active           -                  29.3.1
root@manager01:~#
```

[↑ Sommaire :](#)

4 - Configuration du stockage NFS

NFS : Network File System

4.1 Installation du serveur NFS avec la commande : `apt install -y nfs-kernel-server`

```
root@serveurNFS:~# apt install -y nfs-kernel-server
nfs-kernel-server est déjà la version la plus récente (1:2.8.3-1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@serveurNFS:~#
```

4.2 Vérification du service avec la commande : `systemctl status nfs-kernel-server`

```
root@serveurNFS:~# systemctl status nfs-kernel-server
● nfs-server.service - NFS server and services
   Loaded: loaded (/usr/lib/systemd/system/nfs-server.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2026-04-01 10:32:36 CEST; 55s ago
  Invocation: 30338a45b76244a8b59ac54fc89b6ac9
     Docs: man:rpc.nfsd(8)
           man:exportfs(8)
   Main PID: 2250 (code=exited, status=0/SUCCESS)
  Mem peak: 1.8M
    CPU: 17ms

avril 01 10:32:36 serveurNFS systemd[1]: Starting nfs-server.service - NFS server and services...
avril 01 10:32:36 serveurNFS exportfs[2248]: exportfs: can't open /etc/exports for reading
avril 01 10:32:36 serveurNFS sh[2251]: nfsdctl: lockd configuration failure
avril 01 10:32:36 serveurNFS systemd[1]: Finished nfs-server.service - NFS server and services.
root@serveurNFS:~#
```

4.3 Création du dossier de partage avec la commande : `mkdir -p /data/docker-volumes` et attribution des droits d'accès avec la commande `chmod 777 /data/docker-volumes`

```
root@serveurNFS:~# mkdir -p /data/docker-volumes
root@serveurNFS:~# chmod 777 /data/docker-volumes
root@serveurNFS:~#
```

Tout le monde peut lire et écrire dans ce dossier — nécessaire pour que tous les workers puissent y accéder.

Qui	Permissions	Peut faire
root	7 (rwx)	Lire, écrire, exécuter
groupe	7 (rwx)	Lire, écrire, exécuter
autres	7 (rwx)	Lire, écrire, exécuter

4.4 Edition du fichier de configuration `/etc/exports` avec la commande : `nano /etc/exports`

C'est le fichier de configuration NFS qui définit quels dossiers sont partagés et qui peut y accéder.

```
# Configuration des exports NFS
/data/docker-volumes 192.168.163.0/24(rw,sync,no_subtree_check,no_root_squash)
```

Décomposition :

Élément	Rôle
<code>/data/docker-volumes</code>	Dossier à partager sur le réseau
<code>192.168.163.0/24</code>	Réseau autorisé à accéder au partage — toutes vos VMs
<code>rw</code>	read/write — lecture et écriture autorisées
<code>sync</code>	Les données sont écrites immédiatement sur le disque — plus sûr
<code>no_subtree_check</code>	Désactive la vérification des sous-dossiers — améliore les performances
<code>no_root_squash</code>	L'utilisateur root des clients garde ses droits root sur le partage — nécessaire pour Docker

4.5 Application des exports avec la commande : `exportfs -ra`

Cette commande dit au serveur NFS : "relis ta configuration et applique-la maintenant" sans avoir besoin de redémarrer le service.

4.6 Redémarrage du service NFS avec la commande : `systemctl restart nfs-kernel-server`

```

● nfs-server.service - NFS server and services
   Loaded: loaded (/usr/lib/systemd/system/nfs-server.service; enabled; preset: enabled)
   Active: active (exited) since Wed 2026-04-01 10:50:55 CEST; 8s ago
  Invocation: 391f9d3705f545eb8f9ed81e93f63a14
     Docs: man:rpc.nfsd(8)
           man:exportfs(8)
   Process: 2543 ExecStartPre=/usr/sbin/exportfs -r (code=exited, status=0/SUCCESS)
   Process: 2545 ExecStart=/bin/sh -c /usr/sbin/nfsdctl autostart || /usr/sbin/rpc.nfsd (code=exited, status=0/SUCCESS)
  Main PID: 2545 (code=exited, status=0/SUCCESS)
 Mem peak: 1.8M
    CPU: 16ms

avril 01 10:50:55 serveurNFS systemd[1]: Starting nfs-server.service - NFS server and services...
avril 01 10:50:55 serveurNFS sh[2546]: nfsdctl: lockd configuration failure
avril 01 10:50:55 serveurNFS systemd[1]: Finished nfs-server.service - NFS server and services.

```

4.7 Vérification de l'export avec la commande : **exportfs -v**

```

root@serveurNFS:~# exportfs -v
/data/docker-volumes
192.168.163.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,no_root_squash,no_all_squash)
root@serveurNFS:~#

```

4.8 Installation du client NFS sur les workers avec la commande : **apt install -y nfs-common**

nfs-common : Paquet client NFS qui permet à une machine de "monter" des partages NFS distants

```

root@worker01:~# apt install -y nfs-common
nfs-common est déjà la version la plus récente (1:2.8.3-1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 0
root@worker01:~#

```

4.9 Création du point de montage sur les workers avec la commande : **mkdir -p /mnt/nfs**

/mnt/nfs : Point de montage (dossier local) où le partage NFS sera accessible

4.10 Monter le partage sur les workers avec la commande : **mount 192.168.163.30:/data/docker-volumes /mnt/nfs**

```

root@worker01:~# mkdir -p /mnt/nfs
root@worker01:~# mount 192.168.163.30:/data/docker-volumes /mnt/nfs
root@worker01:~# |

```

4.11 Vérification du montage avec la commande : **df -h | grep nfs**

```

root@worker01:~# df -h | grep nfs
192.168.163.30:/data/docker-volumes 48G 2,0G 44G 5% /mnt/nfs
root@worker01:~#

```

```

root@worker02:~# df -h | grep nfs
192.168.163.30:/data/docker-volumes 48G 2,0G 44G 5% /mnt/nfs

```

```
root@worker03:~# df -h | grep nfs
192.168.163.30:/data/docker-volumes      48G   2,0G   44G   5% /mnt/nfs
root@worker03:~#
```

4.12 Rendre permanent les montages sur les 3 workers avec la commande : `echo "192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0" >> /etc/fstab`

```
root@worker01:~# echo "192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0" >> /etc/fstab
```

/etc/fstab : Fichier de configuration des montages automatiques qui liste tous les systèmes de fichiers à monter automatiquement au démarrage

```
GNU nano 8.4 /etc/fstab *
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# systemd generates mount units based on this file, see systemd.mount(5).
# Please run 'systemctl daemon-reload' after making changes here.
#
# <file system> <mount point> <type> <options> <dump> <pass>
# / was on /dev/sda1 during installation
UUID=e2104f89-db74-4eb7-aed7-a1d4c39c365a / ext4 errors=remount-ro 0 1
# swap was on /dev/sda5 during installation
UUID=ef3ffdd5-04bb-4935-9821-dcb29b52357c none swap sw 0 0
/dev/sr0 /media/cdrom0 udf,iso9660 user,noauto 0 0
# Point de montage
192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0
```

4.12 Vérification du fichier mis à jour avec la commande : `cat /etc/fstab | grep nfs`

```
root@worker01:~# cat /etc/fstab | grep nfs
192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0
```

```
root@worker02:~# cat /etc/fstab | grep nfs
192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0
```

```
root@worker03:~# cat /etc/fstab | grep nfs
192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0
```

4.13 Vérification finale du NFS en créant un fichier test sur le worker 1 avec la commande : `echo "test NFS" > /mnt/nfs/test.txt` et en vérifiant que le fichier est lisible depuis le worker 2 avec la commande `cat /mnt/nfs/test.txt`

```
root@worker01:~# echo "test NFS" > /mnt/nfs/test.txt
```

```
root@worker02:~# cat /mnt/nfs/test.txt
test NFS
```



```
root@worker03:~# cat /mnt/nfs/test.txt
test NFS
```

4.14 Récapitulatif de la configuration NFS

```

Serveur NFS (192.168.163.30)
  └─ /data/docker-volumes  ✓ partagé sur 192.168.163.0/24

Worker 1 (192.168.163.20)  →  /mnt/nfs monté  ✓
Worker 2 (192.168.163.21)  →  /mnt/nfs monté  ✓
Worker 3 (192.168.163.22)  →  /mnt/nfs monté  ✓

/etc/fstab configuré sur les 3 workers  ✓
Test de partage validé  ✓

```

[↑ Sommaire :](#)

5 🚀 - Déploiement des services : Registry, MariaDB, PHP, Nginx & VSCode

5.1 Création du fichier stack.yml avec la commande : **nano**

/root/stack.yml

```
root@manager01:~# nano /root/stack.yml
root@manager01:~#
```

YAML signifie **"YAML Ain't Markup Language"** — c'est un format de configuration lisible par l'humain, utilisé massivement dans le monde DevOps pour décrire des infrastructures. C'est l'implémentation Docker du concept **"Infrastructure as Code"** — toute votre infrastructure est décrite dans un seul fichier texte versionnable sur GitHub. Un collègue peut reprendre votre projet, lancer `docker stack deploy -c stack.yml monapp` et avoir exactement la même infrastructure que vous en quelques secondes.

Voici le fichier stack.yml :


```
C: > Users > leonc > Desktop > ! stack.yml
1  version: "3.8"
2
3  services:
4
5      registry:
6          image: registry:2
7          ports:
8              - "5000:5000"
9          volumes:
10             - /mnt/nfs/registry:/var/lib/registry
11          deploy:
12              replicas: 1
13              placement:
14                  constraints:
15                      - node.role == worker
16          networks:
17              - swarm_network
18
19      mariadb:
20          image: mariadb:latest
21          environment:
22              - MYSQL_ROOT_PASSWORD=rootpassword
23              - MYSQL_DATABASE=swarmdb
24              - MYSQL_USER=swarmuser
25              - MYSQL_PASSWORD=swarmpassword
26          volumes:
27              - /mnt/nfs/mariadb:/var/lib/mysql
28          deploy:
29              replicas: 1
30              placement:
31                  constraints:
32                      - node.role == worker
33          networks:
34              - swarm_network
35
```

```
36 php:
37   image: php:8.2-apache2
38   ports:
39     - "8080:80"
40   volumes:
41     - /mnt/nfs/php:/var/www/html
42   deploy:
43     replicas: 2
44     placement:
45       constraints:
46         - node.role == worker
47   networks:
48     - swarm_network
49
50 nginx:
51   image: nginx:latest
52   ports:
53     - "80:80"
54   volumes:
55     - /mnt/nfs/nginx:/usr/share/nginx/html
56   deploy:
57     replicas: 2
58     placement:
59       constraints:
60         - node.role == worker
61   networks:
62     - swarm_network
63
```

```
63
64   vscode:
65     image: lscr.io/linuxserver/code-server:latest
66     ports:
67       - "8443:8443"
68     environment:
69       - PUID=1000
70       - PGID=1000
71       - TZ=Europe/Paris
72       - PASSWORD=vscodepassword
73     volumes:
74       - /mnt/nfs/vscode:/config
75     deploy:
76       replicas: 1
77       placement:
78         constraints:
79           - node.role == worker
80     networks:
81       - swarm_network
82
83   networks:
84     swarm_network:
85       driver: overlay
```

Vérification de la syntaxe du fichier avec la commande : `docker stack config -c /root/stack.yml`

```

root@manager01:~# docker stack config -c /root/stack.yml
version: "3.8"
services:
  mariadb:
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == worker
    environment:
      MYSQL_DATABASE: swarmdb
      MYSQL_PASSWORD: swarmpassword
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_USER: swarmuser
    image: mariadb:latest
    networks:
      swarm_network: null
    volumes:
      - type: bind
        source: /mnt/nfs/mariadb
        target: /var/lib/mysql

```

5.2 Explications du fichier stack.yml

- **Services :**

Service	Image	Port	Réplicas
Registry	registry:2	5000	1
MariaDB	mariadb:latest	—	1
PHP	php:8.2-apache	8080	2
Nginx	nginx:latest	80	2
VSCode	codercom/code-server	8443	1

- **Définition du nombre de réplicas :**

Services sans état (stateless) → plusieurs répliques OK
 Nginx, PHP, API...
 → chaque requête est indépendante

Services avec état (stateful) → 1 réplique prudent
 MariaDB, Registry...
 → les données doivent rester cohérentes

- **Exposition des différents service :**

Service	Port	Pourquoi ce port
Registry	5000	Port officiel et standard du registry Docker — convention universelle
PHP	8080	Port alternatif HTTP — le 80 est déjà pris par Nginx
Nginx	80	Port HTTP standard — c'est la porte d'entrée principale
VSCode	8443	Port alternatif HTTPS — convention de code-server

- **MariaDB :**

Pourquoi pas de port pour MariaDB ?

MariaDB n'a **pas besoin d'être accessible depuis l'extérieur** — elle est uniquement accessible par les autres conteneurs via le réseau overlay interne `swarm_network`.

✗ Mauvaise pratique :

Internet → port 3306 → MariaDB
 (n'importe qui peut tenter de se connecter)

✓ Bonne pratique :

Internet → Nginx (port 80) → PHP → MariaDB (réseau interne)
 (MariaDB n'est jamais exposée directement)

C'est une règle de sécurité fondamentale — **on n'expose jamais une base de données directement sur internet.**

- **Flux complet des requêtes :**

Utilisateur

↓ port 80

Nginx (reverse proxy)

↓ réseau overlay interne

PHP (port 8080)

↓ réseau overlay interne

MariaDB (pas de port exposé)

↓

Données stockées sur NFS

- Chemin de stockage :

Le même dossier physique `/data/docker-volumes/nginx` sur le serveur NFS est vu sous deux noms différents selon où on se trouve :

Serveur NFS	→	<code>/data/docker-volumes/nginx</code>	(le vrai dossier sur le disque)
Worker	→	<code>/mnt/nfs/nginx</code>	(le même dossier vu via NFS)
Conteneur	→	<code>/usr/share/nginx/html</code>	(le même dossier vu dans Docker)

C'est comme un fichier partagé Google Drive :

Google Drive	→	stockage réel dans le cloud
Votre PC	→	<code>C:\Users\leon\Google Drive\fichier.txt</code>
Votre téléphone	→	<code>/storage/GoogleDrive/fichier.txt</code>

- C'est quoi un réseau overlay ?

Un réseau overlay est un **réseau virtuel qui s'étend sur tous les nœuds du cluster Swarm**. Il permet aux conteneurs de se parler entre eux comme s'ils étaient sur la même machine, même s'ils tournent sur des workers différents.

Services avec 2 réplicas → résistants à la perte d'un worker

Service	Réplicas	Perte d'un worker
Nginx	2	✅ L'autre réplica prend le relais
PHP	2	✅ L'autre réplica prend le relais

Swarm redistribue automatiquement le réplica perdu sur un worker disponible.

Services avec 1 réplica → coupure temporaire

Service	Réplicas	Perte du worker qui l'héberge
Registry	1	⚠️ Coupure le temps que Swarm le redémarre ailleurs
MariaDB	1	⚠️ Coupure temporaire
VSCode	1	⚠️ Coupure temporaire

Swarm va quand même **redémarrer automatiquement** ces conteneurs sur un autre worker, mais il y aura un délai de quelques secondes à quelques minutes.

5.2 Création des dossiers NFS sur le serveur NFS avec les commandes :

- `mkdir -p /data/docker-volumes/registry`
- `mkdir -p /data/docker-volumes/mariadb`
- `mkdir -p /data/docker-volumes/php`
- `mkdir -p /data/docker-volumes/nginx`
- `mkdir -p /data/docker-volumes/vscode`

```
root@serveurNFS:~# mkdir -p /data/docker-volumes/registry
root@serveurNFS:~# mkdir -p /data/docker-volumes/mariadb
root@serveurNFS:~# mkdir -p /data/docker-volumes/php
root@serveurNFS:~# mkdir -p /data/docker-volumes/nginx
root@serveurNFS:~# mkdir -p /data/docker-volumes/vscode
```

5.3 Attribution des droits d'accès avec la commande : `chmod -R 777 /data/docker-volumes/`

```

root@serveurNFS:~# chmod -R 777 /data/docker-volumes
root@serveurNFS:~# ls -la /data/docker-volumes
total 32
drwxrwxrwx 7 root root 4096 1 avril 14:05 .
drwxrwxr-x 3 root root 4096 1 avril 10:45 ..
drwxrwxrwx 2 root root 4096 1 avril 14:05 mariadb
drwxrwxrwx 2 root root 4096 1 avril 14:05 nginx
drwxrwxrwx 2 root root 4096 1 avril 14:05 php
drwxrwxrwx 2 root root 4096 1 avril 14:05 registry
-rwxrwxrwx 1 root root 9 1 avril 11:27 test.txt
drwxrwxrwx 2 root root 4096 1 avril 14:05 vscode
root@serveurNFS:~#

```

5.4 Vérification du cluster avec la commande : `docker node ls`

```

root@manager01:~# docker node ls
ID                HOSTNAME        STATUS        AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqepthkjserhndekyg847y * manager01      Ready         Active           Leader            29.3.1
y293owicg2mi7czcfltcquwmw manager02      Ready         Active           Reachable         29.3.1
ux86nv0shfyvaf0jxkjjt91u5 manager03      Ready         Active           Reachable         29.3.1
z12wbrmij3sr3kssnucp17o3 worker01       Ready         Active           29.3.1
8ifbk2m8yvhmtalyf11bg0a2t worker02       Ready         Active           29.3.1
bggkrw9zvodb86uu8ja5bq1nl worker03       Ready         Active           29.3.1
root@manager01:~# _

```

5.5 Lancer le déploiement avec la commande : `docker stack deploy -c /root/stack.yml swarm`

```

root@manager01:~# docker stack deploy -c /root/stack.yml swarm
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network swarm_swarm_network
Creating service swarm_registry
Creating service swarm_mariadb
Creating service swarm_php
Creating service swarm_nginx
Creating service swarm_vscode
root@manager01:~# |

```

5.6 Vérification des services avec la commande : `docker service ls`

```

root@manager01:~# docker service ls
ID                NAME                MODE                REPLICAS        IMAGE                PORTS
4wrfwv2dqy3f     swarm_mariadb       replicated          1/1             mariadb:latest
soagryla9jg1     swarm_nginx         replicated          2/2             nginx:latest
x1q3oqb6076i     swarm_php           replicated          2/2             php:8.2-apache
uxxmxxwedjaq     swarm_registry      replicated          1/1             registry:2
nnnnop53uk7y     swarm_vscode        replicated          1/1             lscr.io/linuxserver/code-server:latest
root@manager01:~#

```

Vérification des différents services :

- **Nginx** → `http://192.168.163.20`
- **PHP** → `http://192.168.163.20:8080`
- **VSCode** → `http://192.168.163.20:8443`
- **Registry** → `http://192.168.163.20:5000/v2/_catalog`

- Nginx :

403 Forbidden

nginx/1.29.7

- Apache :

Forbidden

You don't have permission to access this resource.

Apache/2.4.66 (Debian) Server at 192.168.163.20 Port 8080

- Vscode :

Welcome to code-server

Please log in below. Password was set from \$PASSWORD.

- Registry :

```

JSON  Données brutes  En-têtes
Enregistrer Copier Tout réduire Tout développer Filtre le JSON
repositories: []

```

pk tout est en .20 ?

Bonne question ! C'est la magie du **routing mesh** de Docker Swarm. 🐳

Même si tes conteneurs tournent sur worker01, worker02 ou worker03, **n'importe quelle IP du cluster répond sur les ports publiés**. Tu aurais pu taper :

- `http://192.168.163.20` (worker01)
- `http://192.168.163.21` (worker02)
- `http://192.168.163.22` (worker03)

Et tu aurais obtenu exactement le même résultat ! Swarm redirige automatiquement la requête vers le bon conteneur, peu importe le nœud que tu interrogues.

C'est ce qu'on appelle le **ingress load balancing** — les requêtes sont réparties automatiquement entre les répliques disponibles, de façon transparente.

[↑ Sommaire :](#)

6 👁 - Supervision et monitoring

Pour la supervision on va déployer 5 outils :

- **Portainer** — interface web pour gérer le cluster visuellement
- **Prometheus** — collecte les métriques du cluster
- **Grafana** — affiche les métriques sous forme de dashboards
- **cAdvisor** — exporter qui fournit les métriques par conteneur
- **node-exporter** — exporter qui fournit les métriques système des hôtes

Service	Rôle	Où	Combien
Portainer	UI cluster	Manager	1 réplica
Prometheus	DB métriques	Manager	1 réplica
Grafana	Dashboards	Manager	1 réplica
cAdvisor	Exporter conteneurs	Tous les nœuds	global (6)
node-exporter	Exporter hôtes	Tous les nœuds	global (6)

6.1 Préparation des managers : montage du NFS pour la persistance de Portainer

Portainer doit tourner sur un nœud manager (besoin de l'API Swarm). Pour assurer la persistance de sa configuration en cas de panne, on monte le partage NFS sur les 3 managers : si le manager qui héberge Portainer tombe, Swarm le redéploie sur un autre manager qui retrouve ses fichiers de config sur le stockage partagé.

```

root@manager01:~# mkdir -p /mnt/nfs
root@manager01:~# mount 192.168.163.30:/data/docker-volumes /mnt/nfs
root@manager01:~# df -h | grep nfs
192.168.163.30:/data/docker-volumes 48G 2,1G 44G 5% /mnt/nfs
root@manager01:~# echo "192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0" >> /etc/fstab
root@manager01:~# cat /etc/fstab | grep nfs
192.168.163.30:/data/docker-volumes /mnt/nfs nfs defaults 0 0
root@manager01:~#

```

6.2 Création des dossiers de monitoring sur le serveur NFS

```

root@serveurNFS:~# mkdir -p /data/docker-volumes/portainer
root@serveurNFS:~# chmod 777 /data/docker-volumes/portainer
root@serveurNFS:~# ls -la /data/docker-volumes/
total 36
drwxrwxrwx 8 root      root      4096  4 mai  10:48 .
drwxrwxr-x 3 root      root      4096  1 avril 10:45 ..
drwxrwxrwx 6          999 systemd-journal 4096  4 mai  10:07 mariadb
drwxrwxrwx 2 root      root      4096  1 avril 14:05 nginx
drwxrwxrwx 2 root      root      4096  1 avril 14:05 php
drwxrwxrwx 2 root      root      4096  4 mai  10:48 portainer
drwxrwxrwx 2 root      root      4096  1 avril 14:05 registry
-rwxrwxrwx 1 root      root         9  1 avril 11:27 test.txt
drwxrwxrwx 9 servernfs servernfs 4096  1 avril 14:21 vscode
root@serveurNFS:~#

```

Prometheus tout seul ne collecte rien. C'est juste une base de données de séries temporelles qui « scrape » (interroge) des **exporters** déployés ailleurs. Pour avoir quelque chose d'intéressant à montrer dans Grafana, il faut 2 exporters de plus :

- **cAdvisor** — fournit les métriques de chaque **conteneur** (CPU/RAM/réseau par service)
- **node-exporter** — fournit les métriques de chaque **hôte** (CPU/RAM/disque/réseau de chaque VM)

Les deux sont déployés en `mode: global`, ce qui veut dire **une instance par nœud du cluster** (donc 6 instances pour chaque exporter — sur les 3 managers + 3 workers).

6.2 Création des dossiers de monitoring sur le serveur NFS

```
root@serveurNFS:~# mkdir -p /data/docker-volumes/prometheus/config
root@serveurNFS:~# mkdir -p /data/docker-volumes/prometheus/data
root@serveurNFS:~# mkdir -p /data/docker-volumes/grafana
root@serveurNFS:~# chmod -R 777 /data/docker-volumes/prometheus
root@serveurNFS:~# chmod -R 777 /data/docker-volumes/grafana
root@serveurNFS:~# ls -la /data/docker-volumes/
total 44
drwxrwxrwx 10 root      root      4096  4 mai  10:57 .
drwxrwxr-x  3 root      root      4096  1 avril 10:45 ..
drwxrwxrwx  2 root      root      4096  4 mai  10:57 grafana
drwxrwxrwx  6          999 systemd-journal 4096  4 mai  10:07 mariadb
drwxrwxrwx  2 root      root      4096  1 avril 14:05 nginx
drwxrwxrwx  2 root      root      4096  1 avril 14:05 php
drwxrwxrwx  2 root      root      4096  4 mai  10:48 portainer
drwxrwxrwx  4 root      root      4096  4 mai  10:57 prometheus
drwxrwxrwx  2 root      root      4096  1 avril 14:05 registry
-rwxrwxrwx  1 root      root         9  1 avril 11:27 test.txt
drwxrwxrwx  9 servernfs servernfs 4096  1 avril 14:21 vscode
root@serveurNFS:~#
```

6.3 Création du fichier de configuration Prometheus

/data/docker-volumes/prometheus/config/prometheus.yml *

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: docker-swarm

scrape_configs:
  # Prometheus se scrape lui-même (méta-monitoring)
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # cAdvisor : métriques de chaque conteneur
  # tasks.cadvisor = entrée DNS Swarm qui retourne les IPs des 6 instances
  - job_name: 'cadvisor'
    dns_sd_configs:
      - names:
          - 'tasks.cadvisor'
        type: 'A'
        port: 8080

  # node-exporter : métriques système des hôtes
  - job_name: 'node-exporter'
    dns_sd_configs:
      - names:
          - 'tasks.node-exporter'
        type: 'A'
        port: 9100

```

Explication du `tasks.<service>` : entrée DNS magique de Docker Swarm qui retourne automatiquement les IPs de toutes les instances d'un service. Comme cAdvisor et node-exporter sont en mode global (1 instance par nœud), Prometheus découvre et scrape automatiquement les 6 cibles de chaque service.

6.4 Création du fichier monitoring.yml sur manager01

```

GNU nano 8.4 /root/monitoring.yml *
version: "3.8"

services:
# =====
# PORTAINER - UI de gestion du cluster
# =====
  portainer:
    image: portainer/portainer-ce:latest
    ports:
      - "9000:9000"
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - /mnt/nfs/portainer:/data
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == manager
    networks:
      - monitoring

# =====
# PROMETHEUS - collecte des métriques
# =====
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - /mnt/nfs/prometheus/config:/etc/prometheus
      - /mnt/nfs/prometheus/data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.role == manager
    networks:
      - monitoring

```

```

# =====
# GRAFANA - dashboards
# =====
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
  volumes:
    - /mnt/nfs/grafana:/var/lib/grafana
  deploy:
    replicas: 1
    placement:
      constraints:
        - node.role == manager
  networks:
    - monitoring

# =====
# cADVISOR - exporter conteneurs (1 par nœud)
# =====
cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
    - /dev/disk:/dev/disk:ro
  deploy:
    mode: global
  networks:
    - monitoring

```

```
# =====
# NODE-EXPORTER - exporter hôtes (1 par nœud)
# =====
node-exporter:
  image: prom/node-exporter:latest
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /:/rootfs:ro
  command:
    - '--path.procfs=/host/proc'
    - '--path.sysfs=/host/sys'
    - '--path.rootfs=/rootfs'
    - '--collector.filesystem.mount-points-exclude=^(sys|proc|dev|host|e>
  deploy:
    mode: global
  networks:
    - monitoring

networks:
  monitoring:
    driver: overlay
    attachable: true
```

- `replicas: 1 + node.role == manager` : Portainer/Prometheus/Grafana tournent sur un seul manager (n'importe lequel). Si ce manager crash, Swarm les redéploie sur un autre manager et grâce au NFS la config est conservée.
- `mode: global` : cAdvisor et node-exporter sont lancés une fois par nœud (6 instances pour chaque). Indispensable car ils mesurent l'état local de chaque hôte.
- `zcube/cadvisor` au lieu de `gcr.io/cadvisor/cadvisor` : on a basculé sur un mirror Docker Hub car le pull depuis Google Container Registry était trop lent / bloqué.
- **Réseau monitoring** : réseau overlay isolé du `swarm_network` applicatif pour bien séparer les flux.

6.5 Vérification de la syntaxe et déploiement

```
root@manager01:~# docker stack config -c /root/monitoring.yml
```

```
root@manager01:~# docker stack deploy -c /root/monitoring.yml monitoring
Since --detach=false was not specified, tasks will be created in the backgro
und.
In a future release, --detach=false will become the default.
Creating network monitoring_monitoring
Creating service monitoring_grafana
Creating service monitoring_cadvisor
Creating service monitoring_node-exporter
Creating service monitoring_portainer
Creating service monitoring_prometheus
root@manager01:~#
```



```

root@manager01:~# docker service ls
ID                NAME                                MODE                REPLICAS    IMAGE                                           PORTS
2z9jlqdc2vpe      monitoring_cadvisor                global              6/6          zcube/cadvisor:latest
9p0kykt82cza      monitoring_grafana                 replicated          1/1          grafana/grafana:latest                       *:3000->3000/tcp
n6su91xb8utl      monitoring_node-exporter           global              6/6          prom/node-exporter:latest
ikrprdhtdgox      monitoring_portainer               replicated          1/1          portainer/portainer-ce:latest                *:9000->9000/tcp, *:9443->9443/tcp
l09ty40bovn2      monitoring_prometheus              replicated          1/1          prom/prometheus:latest                      *:9090->9090/tcp
4wrfwv2dqy3f      swarm_mariadb                     replicated          1/1          mariadb:latest
soagryla9jg1      swarm_nginx                       replicated          2/2          nginx:latest                                *:80->80/tcp
xlq3oqb6076i      swarm_php                          replicated          2/2          php:8.2-apache                             *:8080->80/tcp
uwxmxkwedjaq      swarm_registry                    replicated          1/1          registry:2                                 *:5000->5000/tcp
nnnop53uk7y       swarm_vscode                      replicated          1/1          lscr.io/linuxserver/code-server:latest      *:8443->8443/tcp
root@manager01:~#

```

6.6 Accès aux interfaces web

Grâce au routing mesh de Swarm, n'importe quelle IP du cluster fonctionne :

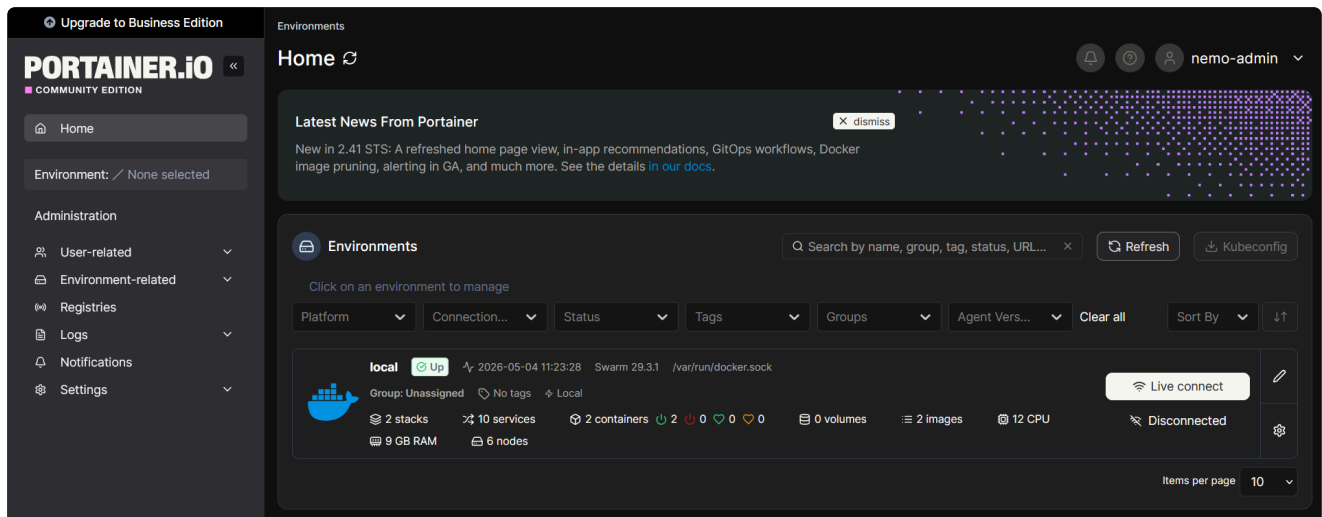
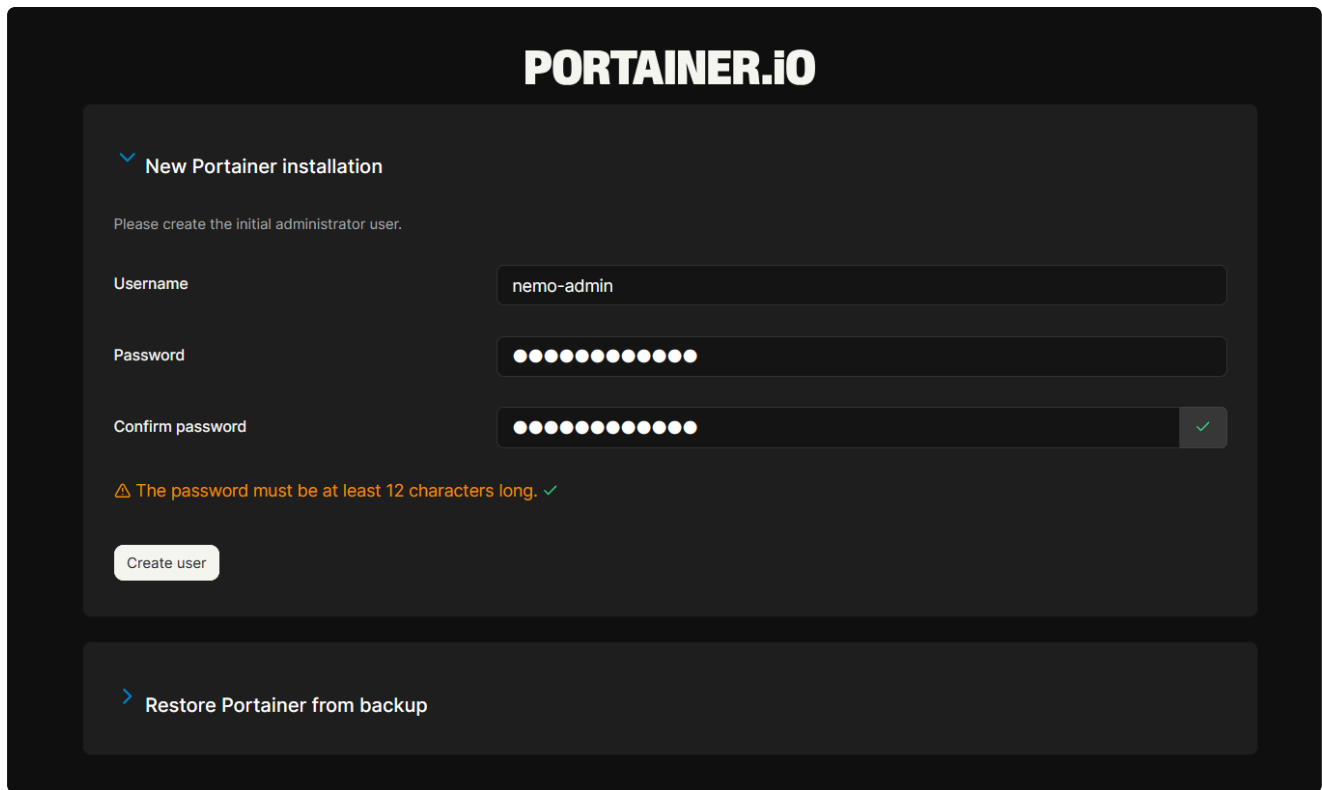
Outil	URL
Portainer	http://192.168.163.10:9000
Prometheus	http://192.168.163.10:9090
Grafana	http://192.168.163.10:3000

Portainer

Portainer est une **interface web de gestion de conteneurs Docker**. Au lieu de taper des commandes en CLI (`docker service ls`, `docker stack deploy`, etc.), Portainer permet de tout faire visuellement depuis un navigateur.

Ce qu'il apporte concrètement :

- Vue graphique du cluster : tous les nœuds, services, conteneurs, volumes et réseaux en un coup d'œil
- Cluster visualizer : représentation visuelle de la répartition des conteneurs sur les nœuds
- Lecture des logs en direct, statistiques CPU/RAM par conteneur
- Scaling d'un service en 1 clic (passer de 2 à 5 répliques par exemple)
- Déploiement, mise à jour ou suppression de stacks sans toucher à la ligne de commande



Grafana

Grafana est une **plateforme de visualisation de données** open source, spécialisée dans l'affichage de séries temporelles sous forme de **dashboards** interactifs.

Caractéristique principale : Grafana ne stocke pas les données lui-même — il se connecte à des **datasources** externes (Prometheus, InfluxDB, Elasticsearch, MySQL, etc.) et affiche leurs données en temps réel.

Composants clés :

Élément	Rôle
Datasource	source de données à interroger (dans notre cas : Prometheus)
Dashboard	tableau de bord composé de plusieurs panels

Élément	Rôle
Panel	un graphique, une jauge, une carte de chaleur, un tableau, etc.
Variable	filtre dynamique (ex : sélecteur d'hôte ou de conteneur en haut du dashboard)

Bibliothèque communautaire : Grafana met à disposition un catalogue public de dashboards préconçus identifiables par un ID numérique. Au lieu de construire ses dashboards de zéro, on peut importer ceux de la communauté en saisissant simplement leur ID :

ID	Nom	Source de données
1860	Node Exporter Full	node-exporter
14282	cAdvisor exporter	cAdvisor
11600	Docker Swarm Nodes	cAdvisor + node-exporter

Rôle dans le projet : couche de visualisation par-dessus Prometheus — c'est l'interface humaine du monitoring.

Dashboards > Import dashboard

+

▼

Import dashboard

Import dashboard from file or Grafana.com

Importing dashboard from [Grafana.com](https://grafana.com)

Published by

rfmoz

Updated on

2026-04-11 10:15:40

Options

Name

Node Exporter Full

Folder

Dashboards

Unique identifier (UID)

The unique identifier (UID) of a dashboard can be used to uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.

rYdddIPWk

Change uid

Import

Cancel



Prometheus

Prometheus est un **système de collecte et de stockage de métriques** créé par SoundCloud, devenu standard de l'industrie pour le monitoring d'infrastructures cloud-native.

Principe de fonctionnement : Prometheus est un système « **pull-based** » — au lieu d'attendre que les machines surveillées lui envoient leurs données, c'est lui qui va

régulièrement les interroger (on parle de **scraping**). Il interroge des **exporters** : petits programmes qui exposent les métriques d'une cible donnée sur une URL HTTP.

Composants utilisés dans le projet :

Composant	Rôle
Prometheus (server)	scrape les exporters toutes les 15 secondes et stocke les données dans une base TSDB
node-exporter	exporter qui expose les métriques système d'une VM (CPU, RAM, disque, réseau)
cAdvisor	exporter qui expose les métriques de chaque conteneur (CPU, RAM, I/O par conteneur)

TSDB (Time Series DataBase) : base de données spécialisée dans le stockage de séries temporelles — chaque métrique est associée à un horodatage et à des labels (`instance` , `job` , etc.) pour pouvoir être interrogée et agrégée efficacement.

Langage de requête : Prometheus utilise **PromQL**, un langage qui permet d'agréger, filtrer, calculer des taux, des moyennes, etc. sur les séries temporelles.

Rôle dans le projet : moteur de collecte des métriques de tout le cluster. Prometheus sait découvrir automatiquement les nouveaux conteneurs grâce au DNS service discovery de Docker Swarm (`tasks.cadvisor` , `tasks.node-exporter`).

Pour la supervision on va déployer 3 outils :

- **Portainer** — interface web pour gérer le cluster visuellement
- **Prometheus** — collecte les métriques du cluster
- **Grafana** — affiche les métriques sous forme de dashboards

7 - Test PCA/PRA

Cette section regroupe les tests de validation du Plan de Continuité d'Activité (PCA) et du Plan de Reprise d'Activité (PRA) du cluster. Chaque test simule un type de panne différent et vérifie que le cluster réagit conformément aux mécanismes de résilience décrits dans les sections précédentes (réplication des services, élection Raft, persistance NFS, routing mesh).

7.1 Test du self-healing : kill d'un conteneur applicatif

7.1.1 État du service `swarm_nginx` avant le test : 2 réplicas running, équilibrés sur deux workers différents

mar. 05 mai 2026 10:13:11 CEST

ID	CURRENT STATE	NAME	ERROR	PORTS	IMAGE	NODE	DESIRED STATE
qyez6w3bw4m3bvzybs96d6hm	Running 42 seconds ago	swarm_nginx.1			nginx:latest@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18	worker02	Running
uvp9eovuqutmv3z7t95j7e80m	Running 5 minutes ago	swarm_nginx.2			nginx:latest@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18	worker03	Running

7.1.2 Simulation d'une panne avec la commande docker kill du conteneur Nginx hébergé sur worker02

```

root@worker02:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS          NAMES
83a5884ff096   nginx:latest                        "/docker-entrypoint. ..." About a minute ago Up About a minute 80/tcp          swarm_nginx.1.qyez6w3bw4m3bvzybs96d6hm
3d88a02fcf33   zcube/cadvisor:latest              "/usr/bin/cadvisor - ..." 6 minutes ago    Up 6 minutes (healthy) 8080/tcp        monitoring_cadvisor.8ifbk2m8yvh
mtalyfl1bg0a2t.iuvcw6yhucdl6mggyn62m0jau
0678d3b48376   prom/node-exporter:latest          "/bin/node_exporter - ..." 6 minutes ago    Up 6 minutes          9100/tcp        monitoring_node-exporter.8ifbk2m8yvh
m8yvhmtalyfl1bg0a2t.verjqgz5kcuiwz0qim8prn6z
root@worker02:~# docker kill 83a5884ff096
83a5884ff096
root@worker02:~#

```

7.1.3 Vérification de la reprise automatique du service moins d'une seconde après la création automatique du conteneur de remplacement par Swarm

mar. 05 mai 2026 10:15:17 CEST

ID	CURRENT STATE	NAME	ERROR	PORTS	IMAGE	NODE	DESIRED STATE
98i63ymlb8fckvu411f539tu0	Starting less than a second ago	swarm_nginx.1			nginx:latest@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18	worker02	Running
uvp9eovuqutmv3z7t95j7e80m	Running 7 minutes ago	swarm_nginx.2			nginx:latest@sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18	worker03	Running

Test 7.2 — Panne d'un worker

Objectif : prouver que la perte d'un nœud worker complet (et pas juste d'un conteneur) est compensée automatiquement par Swarm. Les réplicas hébergés sur le worker tombé sont redéployés sur les workers restants, et le service reste accessible via routing mesh.

7.2.1 Sur manager01 : Etat du cluster

```

root@manager01:~# docker node ls
docker service ps swarm_nginx
ID          HOSTNAME    STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqeptkhkjerhndekyg847y * manager01    Ready     Active           Reachable         29.3.1
y293owicg2mi7czcfltcquwmw manager02    Ready     Active           Reachable         29.3.1
ux86nv0shfyvaf0jxkjjt91u5 manager03    Ready     Active           Leader            29.3.1
z12wbrmij3sr3kssnurcp17o3 worker01     Ready     Active           Ready             29.3.1
8ifbk2m8yvhmtalyfl1bg0a2t worker02     Ready     Active           Ready             29.3.1
bggkrw9zvodb86uu8ja5bq1nl worker03     Ready     Active           Ready             29.3.1
ID          NAME          IMAGE          NODE    DESIRED STATE    CURRENT STATE    ERROR    PORTS
98i63ymlb8fckvu411f539tu0 swarm_nginx.1 nginx:latest   worker02 Running          Running 16 minutes ago
uvp9eovuqutmv3z7t95j7e80m swarm_nginx.2 nginx:latest   worker03 Running          Running 24 minutes ago
rzpqgf70sxcu swarm_php.1   php:8.2-apache worker01 Running          Running 24 minutes ago
31plm0o9z8xp swarm_php.2   php:8.2-apache worker03 Running          Running 24 minutes ago
ps1e047fj95v swarm_registry.1 registry:2     worker03 Running          Running 24 minutes ago
root@manager01:~#

```

7.2.2 On éteint sur Vmware le worker03

```

mar. 05 mai 2026 10:36:54 CEST
=== Nodes ===
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqepthkjsrhndekylg847y * manager01      Ready     Active           Reachable          29.3.1
y293owicg2mi7czcfltcquwmw manager02      Ready     Active           Reachable          29.3.1
ux86nv0shfyvaf0jxkjjt91u5 manager03      Ready     Active           Leader             29.3.1
z12wbrmij3sr3kssnurcp17o3 worker01       Ready     Active           29.3.1
8ifbk2m8yvvhmtalyfl1bg0a2t worker02       Ready     Active           29.3.1
bggkrw9zvodb86uu8ja5bq1nl worker03       Down      Active           29.3.1

=== Services ===
ID                NAME                MODE        REPLICAS    IMAGE                                  PORTS
2z9jldc2vpe      monitoring_cadvisor global        6/5         zcube/cadvisor:latest               *
9p0kykt82cza      monitoring_grafana   replicated   1/1         grafana/grafana:latest              *:3000->3000/tcp
n6su91xb8utl      monitoring_node-exporter global        6/5         prom/node-exporter:latest           *
ikrprdhtdgox      monitoring_portainer replicated    1/1         portainer/portainer-ce:latest       *:9000->9000/tcp, *:9443->9443/tcp
109ty40bovn2      monitoring_prometheus replicated    1/1         prom/prometheus:latest              *:9090->9090/tcp
4wrfwv2dqy3f      swarm_mariadb        replicated   1/1         mariadb:latest                      *
soagryla9jg1      swarm_nginx          replicated   3/2         nginx:latest                        *:80->80/tcp
xlq3oqb6076i      swarm_php            replicated   3/2         php:8.2-apache                      *:8080->80/tcp
uxxmwxwedjaq      swarm_registry       replicated   2/1         registry:2                          *:5000->5000/tcp
nnnop53uk7y       swarm_vscode         replicated   2/1         lscr.io/linuxserver/code-server:latest *:8443->8443/tcp

```

Ce phénomène est la preuve que Swarm est actif. Il préfère démarrer le remplaçant tout de suite, quitte à avoir un instant un réplica de trop, plutôt que d'attendre la confirmation de la mort et risquer une downtime. C'est une stratégie de réconciliation forward-looking.

Test 7.2 — Panne d'un lead manager

7.2.1 Etat du Cluster

```

mar. 05 mai 2026 11:12:59 CEST
=== Nodes ===
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqepthkjsrhndekylg847y * manager01      Ready     Active           Reachable          29.3.1
y293owicg2mi7czcfltcquwmw manager02      Ready     Active           Reachable          29.3.1
ux86nv0shfyvaf0jxkjjt91u5 manager03      Ready     Active           Leader             29.3.1
z12wbrmij3sr3kssnurcp17o3 worker01       Ready     Active           29.3.1
8ifbk2m8yvvhmtalyfl1bg0a2t worker02       Ready     Active           29.3.1
bggkrw9zvodb86uu8ja5bq1nl worker03       Down      Active           29.3.1

```

7.2.2 Etat du Cluster après shutdown de la VM manager03

```

mar. 05 mai 2026 11:14:27 CEST
=== Nodes ===
ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqepthkjsrhndekylg847y * manager01      Ready     Active           Reachable          29.3.1
y293owicg2mi7czcfltcquwmw manager02      Ready     Active           Leader             29.3.1
ux86nv0shfyvaf0jxkjjt91u5 manager03      Down      Active           Unreachable        29.3.1
z12wbrmij3sr3kssnurcp17o3 worker01       Ready     Active           29.3.1
8ifbk2m8yvvhmtalyfl1bg0a2t worker02       Ready     Active           29.3.1
bggkrw9zvodb86uu8ja5bq1nl worker03       Down      Active           29.3.1

```

7.2.1 Etat des services du Cluster après shutdown de la VM manager03

```

root@manager01:~# docker service ls
docker node ls
ID                NAME                MODE        REPLICAS    IMAGE                                  PORTS
2z9jldc2vpe      monitoring_cadvisor global        6/4         zcube/cadvisor:latest               *
9p0kykt82cza      monitoring_grafana   replicated   1/1         grafana/grafana:latest              *:3000->3000/tcp
n6su91xb8utl      monitoring_node-exporter global        6/4         prom/node-exporter:latest           *
ikrprdhtdgox      monitoring_portainer replicated    1/1         portainer/portainer-ce:latest       *:9000->9000/tcp, *:9443->9443/tcp
109ty40bovn2      monitoring_prometheus replicated    1/1         prom/prometheus:latest              *:9090->9090/tcp
4wrfwv2dqy3f      swarm_mariadb        replicated   1/1         mariadb:latest                      *
soagryla9jg1      swarm_nginx          replicated   3/2         nginx:latest                        *:80->80/tcp
xlq3oqb6076i      swarm_php            replicated   3/2         php:8.2-apache                      *:8080->80/tcp
uxxmwxwedjaq      swarm_registry       replicated   2/1         registry:2                          *:5000->5000/tcp
nnnop53uk7y       swarm_vscode         replicated   2/1         lscr.io/linuxserver/code-server:latest *:8443->8443/tcp

ID                HOSTNAME        STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
co8cqepthkjsrhndekylg847y * manager01      Ready     Active           Reachable          29.3.1
y293owicg2mi7czcfltcquwmw manager02      Ready     Active           Leader             29.3.1
ux86nv0shfyvaf0jxkjjt91u5 manager03      Down      Active           Unreachable        29.3.1
z12wbrmij3sr3kssnurcp17o3 worker01       Ready     Active           29.3.1
8ifbk2m8yvvhmtalyfl1bg0a2t worker02       Ready     Active           29.3.1
bggkrw9zvodb86uu8ja5bq1nl worker03       Down      Active           29.3.1
root@manager01:~#

```

Compétences visées par le sujet

Compétence 1 : Administrer et sécuriser les infrastructures systèmes

- Installation et configuration de 7 systèmes Debian (sections 2 et 3)
- Sécurisation des dépôts Docker via clés GPG (section 3.2-3.4)
- Gestion fine des permissions de fichiers (`chmod 0755, 777, 0600` selon le contexte)
- Configuration du service SSH (section 2.6)
- Résolution de conflits de paquets (downgrade `iptables` et `libxtables12` , section 3.7)

Compétence 2 : Administrer et sécuriser les infrastructures virtualisées

- Création de 7 VMs sous VMware Workstation avec ressources adaptées (sections 2.1, 2.3)
- Plan d'adressage IP cohérent et documenté (section 2.5)
- Configuration réseau statique sur chaque VM
- Gestion des cycles de vie des VMs (Power On / Off pour les tests PCA)
- Isolation réseau via réseaux overlay Docker (section 6)

Compétence 3 : Concevoir une solution technique répondant à des besoins d'évolution de l'infrastructure

- Choix d'une **architecture haute disponibilité** (3 managers + 3 workers) au lieu du minimum requis
- **Stockage centralisé NFS** permettant l'ajout de workers sans migration de données
- **Mode global** pour cAdvisor et node-exporter : auto-extensible si on ajoute des nœuds
- **Infrastructure as Code** (`stack.yml` , `monitoring.yml`) versionnable et reproductible
- **Scaling à la demande** prouvé par `docker service scale` lors du test
- **Routing mesh** qui permet de placer les services n'importe où dans le cluster

Compétence 4 : Mettre en œuvre et optimiser la supervision des infrastructures

- Déploiement d'une **stack monitoring complète** (Portainer + Prometheus + Grafana + cAdvisor + node-exporter)
- Configuration de Prometheus avec **DNS service discovery** Swarm (auto-découverte des cibles)
- Import de **dashboards communautaires** Grafana
- Surveillance des **6 nœuds** simultanément (CPU, RAM, disque, réseau, conteneurs)

- Possibilité d'**alerting** (capacité non utilisée mais possible avec Prometheus Alertmanager)

#01